Predictive Learning — Homework 1

Risk Minimization and Model Selection

Name: Shuo-Wun Chiang

ID: 61547101S

Problems 2.7, 2.8, 2.11, and 2.12

Code and data: https://github.com/tc9305/predict_learning/tree/main/HW1

Overview

This report examines risk minimization and model selection through classification and regression experiments. Problems 2.7 and 2.8 use obesity data from two election years to study nearest-neighbor classification, with the number of neighbors selected by leave-one-out cross-validation. Problems 2.11 and 2.12 compare algebraic polynomial and cosine regression models, using a complexity-adjusted criterion for model selection and five-fold cross-validation to evaluate predictive performance.

Problem 2.7: KNN classification across election years

Problem setup

The task is to predict each state's election label using obesity percentage as the single input feature. Following the problem specification, I train a KNN classifier using the 2004 data and evaluate it using the 2000 data.

Data and experimental settings

The supplied election datasets each contain 51 observations: the 50 states and Washington, D.C. Only obesity percentage is used as an input feature. The response is the supplied Democratic (D) or Republican (R) election label. Obesity itself is not the prediction target.

Year	Observations	D	R	Mean obesity (%)
2000	51	21	30	19.10
2004	51	19	32	22.39

The original data order and supplied labels are preserved, including the 2004 Hawaii label R. All election scores therefore evaluate the supplied labels.

Implementation settings

The following code specifies the KNN settings used in this experiment. Problem 2.8 uses the same settings with the training and test years reversed.

Listing 1: KNN configuration used in the experiments.

```
def make_knn(k):
    return KNeighborsClassifier(
        n_neighbors=k,
        weights='uniform',
        metric='euclidean',
        algorithm='brute',
        n_jobs=1
    )
```

Why these settings?

I chose brute-force search because the training dataset contains only 51 observations and one input feature, making direct distance computation practical. Euclidean distance reduces to the absolute difference between two obesity percentages. Uniform voting gives each selected neighbor one vote. The number of neighbors is selected by LOOCV rather than fixed in advance. The setting `n_jobs=1` controls parallel jobs for neighbor search; the experiment also uses `threadpool_limits(limits=1)` to limit numerical-library threads.

Selecting the number of neighbors by LOOCV

As required by the problem, I select k using leave-one-out cross-validation on the 2004 training data. For each candidate k, I hold out one observation, fit the classifier using the other 50 observations, and predict the omitted label. Repeating this for all 51 observations gives one held-out prediction for every training observation.

I evaluate all $k = 1, \ldots, 50$, including both odd and even values. The maximum is 50 because each LOO fit contains only 50 training observations. No observation from the 2000 test set is used to select k. Using 50 observations in each fit retains most of this small training sample, although LOOCV does not guarantee that the selected model will transfer well to another year.

The following code shows how held-out predictions and errors are calculated for each candidate k. Here `x` contains the 2004 obesity percentages, `y` contains their election labels, and `rows` collects the validation scores. Numerical-library threads are limited to one during this procedure.

Listing 2: LOOCV predictions and error calculation for each candidate k.

```
for k in range(1, len(y)):
    predictions = np.empty(len(y), dtype=y.dtype)
    for train, validation in LeaveOneOut().split(x):
        model = make_knn(k).fit(x[train], y[train])
        predictions[validation] = model.predict(x[validation])
    wrong = predictions != y
    rows.append({
        'k': k,
        'wrong_count': int(wrong.sum()),
        'loocv_error': float(wrong.mean())
    })
```

The Boolean array `wrong` records the 0–1 loss: an incorrect label contributes 1, and a correct label contributes 0. Taking its mean gives the resampling error:

$$\widehat{R}_{\mathrm{LOO}}(k) = \frac{1}{51} \sum_{i=1}^{51} \mathbf{1}\{\widehat{y}_{-i,k}(x_i) \neq y_i\}\,. \tag{1}$$

Here $\widehat{y}_{-i,k}$ denotes the classifier fitted without observation i. I select the smallest k attaining the minimum error, as detailed below.

Tie handling

Three distinct types of ties are handled as follows:

1. **Equal class votes.** With uniform voting, D and R can receive the same number of votes when k is even. The current scikit-learn classifier resolves this tie in favor of the first class in its sorted class order. For the labels used here, `classes_` is `['D', 'R']`, so a vote tie predicts D. This is a software convention, not an assumption that D is more likely. With two classes, odd k avoids a class-vote tie, but it does not eliminate equal-distance ties.

2. **Equal neighbor distances.** Equal distances within the selected set do not change a uniform vote if set membership is unchanged. The important boundary case occurs when the kth and $(k+1)$th candidate neighbors are equally distant and have different labels: selecting a different boundary point can change the prediction. My code does not implement a secondary rule such as preferring D or the smallest row index. It uses the neighbor set returned by scikit-learn's brute-force implementation. I preserve the original input order and fix the search backend and thread settings. The reported results use scikit-learn 1.9.1; the software version, input order, backend and thread settings specify the experimental conditions. This does not guarantee that the earliest input row is always selected, or that another backend or software version selects the same tied neighbors.[1]

3. **Equal LOOCV errors for different k.** Candidate values are evaluated in ascending order, $k = 1, \ldots, 50$. The code below returns the first minimum, so the smallest k among the tied candidates is selected. In Problem 2.7, $k = 7, 20, 22$ attain the same minimum error, and this rule selects $k = 7$.

[1] The scikit-learn documentation warns that predictions can depend on training-data order when the kth and $(k+1)$th neighbors are equally distant and have different labels: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html.

Beyond providing a reproducible convention, this rule expresses a preference for more local decisions when the observed validation errors are identical. Compared with $k = 20$ or 22, $k = 7$ bases each prediction on a smaller neighborhood and retains greater sensitivity to local variation. Larger neighborhoods generally smooth decision boundaries and, when sufficiently broad, can be dominated by the overall majority class, which is R in the 2004 training data. However, equal aggregate LOOCV errors do not demonstrate that the larger candidates actually over-smooth local patterns, and smaller neighborhoods can be more sensitive to noise and distance ties. Thus, choosing the smallest tied k preserves a more local decision rule without claiming lower true prediction risk or general superiority over larger k.

Listing 3: Excerpt implementing the tie rule for selecting k.

```
best_position = table['loocv_error'].to_numpy().argmin()
return int(table['k'].to_numpy()[best_position]), table, pd.DataFrame(audit)
```

Refitting and prediction

After choosing k, I fit the final classifier on all 51 observations from 2004 and predict the labels for the 2000 observations. This separates model selection from the final test evaluation. The following code shows the selection, refitting and prediction steps. Here `train` and `test` refer to the 2004 and 2000 data, respectively.

Listing 4: Select k on the training data, refit, and predict the test year.

```
x, y = train[['Obesity']].to_numpy(), train.Election_Result.to_numpy()
k, cv, audit = select_knn(x, y)
with threadpool_limits(limits=1):
    model = make_knn(k).fit(x, y)
    train_prediction = model.predict(x)
    prediction = model.predict(test[['Obesity']].to_numpy())
```

Results

Quantity	Problem 2.7
Training → test year	2004 → 2000
Tied best k values	$7, 20, 22$
Selected k	7
Minimum LOOCV error	$14/51 = 27.45\%$
Training error	$12/51 = 23.53\%$
Test error	$26/51 = 50.98\%$
Test accuracy	$25/51 = 49.02\%$

The minimum LOOCV error is shared by $k = 7, 20, 22$. Applying the smallest-k tie rule gives $k = 7$. This value is selected using only the 2004 data; it is not chosen because it gives the best result on the 2000 test data.

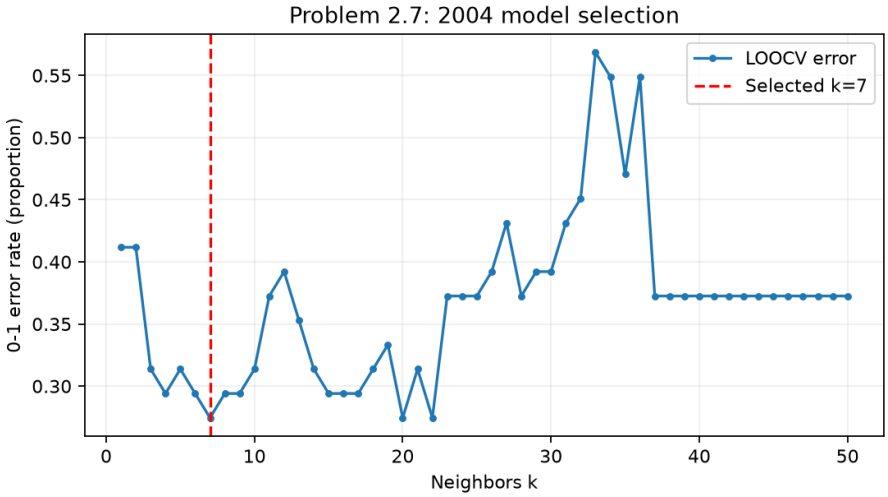


Figure 1: Problem 2.7: LOOCV error on the 2004 training data. The dashed red line marks the selected $k = 7$; $k = 20$ and $k = 22$ attain the same minimum error.

The test confusion matrix has actual D/R labels in its rows and predicted D/R labels in its columns:

$$C_{2.7} = \begin{pmatrix} 17 & 4 \\ 22 & 8 \end{pmatrix}.$$

Thus, 17 of the 21 D observations and 8 of the 30 R observations are correctly classified. The errors consist of 4 D observations predicted as R and 22 R observations predicted as D, totaling 26 mistakes.

Discussion

The test error is 50.98%, which exceeds the minimum LOOCV error of 27.45% by 23.53 percentage points. The LOOCV score evaluates omitted observations from the 2004 sample and is also used to select k. The test error evaluates transfer to the 2000 sample. The minimized LOOCV score is therefore not a guarantee of cross-year prediction quality.

The test accuracy of 49.02% indicates limited predictive performance in this experiment. For context, an always-R classifier based on the majority label in the 2004 training data would correctly classify $30/51 = 58.82\%$ of the supplied 2000 labels. The selected KNN classifier performs worse than this simple baseline. This baseline comparison is made after evaluation and is not used to tune k.

Using obesity alone assumes that states with similar obesity percentages tend to have similar election labels. The large number of R observations predicted as D shows that this assumption does not yield reliable cross-year predictions here. Changes in the obesity distribution and the relationship between obesity and election labels are possible explanations, but these results do not identify a cause. The same states occur in both years, so the test set is not an independent sample of new states. The analysis evaluates the supplied labels and does not establish a causal effect of obesity on voting.

Problem 2.8: Reversing the training and test years

I reuse the two election datasets described in Problem 2.7. For this problem, the 2000 observations are used for training and the 2004 observations for testing. The implementation and tie rules are the same as in Problem 2.7, but LOOCV is repeated using only the new training year to select k independently.

Quantity	Problem 2.8
Training $\rightarrow$ test year	$2000 \rightarrow 2004$
Tied best k values	$7, 9, 16, 24, 25, 26, 27, 28$
Selected k	7
Minimum LOOCV error	$19/51 = 37.25\%$
Training error	$14/51 = 27.45\%$
Test error	$22/51 = 43.14\%$
Test accuracy	56.86%

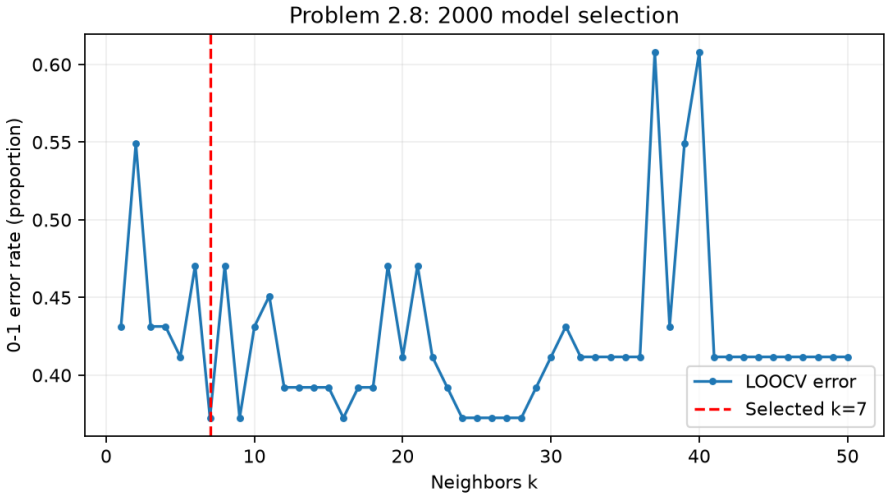


Figure 2: Problem 2.8: LOOCV on the 2000 training data independently selects $k = 7$.

Rows of the test confusion matrix are actual D/R, and columns are predicted D/R:

$$C_{2.8} = \begin{pmatrix} 2 & 17 \\ 5 & 27 \end{pmatrix}.$$

The cross-year error differs from the within-year LOO error. Changes in obesity distributions and label proportions may affect transfer. Selecting the smallest tied k is an explicit tie rule, not proof that smaller k is universally superior. Equal-distance neighbors can also make implementation details relevant. The same states occur in both years, so this is not evaluation on an independent sample of new states. These results do not establish a causal effect of obesity on voting.

Problem 2.11: Regression and model selection

Problem setup and data

This problem compares trigonometric and algebraic polynomial regression. For each family, I fit the coefficients at each candidate order and use the analytic Schwartz criterion to select the order. I then consider whether these selection scores alone establish which family predicts better.

I generate ten observations according to the specified data-generating process:

$$x_i \sim \mathcal{U}(0,1), \qquad y_i = x_i^2 + 0.1x_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, 0.5^2). \tag{2}$$

The noise variance is 0.25, so the standard deviation passed to NumPy is 0.5. The following code fixes the random seed at 42 so that the sample can be reproduced. Both model families use exactly these same observations.

Listing 5: Generating the ten regression observations.

```
rng = np.random.default_rng(42)
x = rng.uniform(0, 1, 10)
noise = rng.normal(0, 0.5, 10)
y = x**2 + 0.1*x + noise
```

This is a newly generated sample from the specified process, rather than the exact observations in the textbook example. The generating curve is shown in the figures for interpretation; it is not used to estimate coefficients or select the order.

Model families and basis construction

Following the problem definition, I use the cosine-only trigonometric basis and the algebraic power basis:

$$f_m^{\cos}(x) = w_0 + \sum_{j=1}^{m} w_j \cos(2\pi j x), \tag{3}$$

$$f_m^{\text{alg}}(x) = \sum_{j=0}^{m} w_j x^j. \tag{4}$$

In the algebraic family, m is the highest polynomial degree. In the cosine family, m is the maximum frequency index. Each family has $m+1$ fitted coefficients, including the constant. At $m = 0$, both reduce to a constant model. The label `'trigonometric'` in the code refers to the cosine-only family above.

The design matrix contains one row for each observation and one column for each basis function. The core construction is:

Listing 6: Constructing the cosine or algebraic design matrix.

```
def design_matrix(x, m, family):
    x = np.asarray(x, dtype=float).reshape(-1)
    if family == 'trigonometric':
        return np.column_stack([
            np.ones(len(x)),
            np.cos(2*np.pi*x[:, None]*np.arange(1, m+1))
        ])
    if family == 'algebraic':
        return np.vander(x, N=m+1, increasing=True)
```

For example, at $m = 2$, the algebraic basis is 1, x, and x^2; the cosine basis is 1, $\cos(2\pi x)$, and $\cos(4\pi x)$. Neither family includes sine terms.

Least-squares fitting and implementation choice

For a fixed family and order, the coefficients are estimated by ordinary least squares:

$$\widehat{\boldsymbol{w}}_m = \arg\min_{\boldsymbol{w}} \|\boldsymbol{y} - A_m \boldsymbol{w}\|_2^2, \qquad \widehat{\boldsymbol{y}} = A_m \widehat{\boldsymbol{w}}_m. \tag{5}$$

Although the basis functions can be nonlinear in x, the model is linear in its coefficients. The numerical fitting step is:

Listing 7: Least-squares estimation for a fixed family and order.

```
a = design_matrix(x, m, family)
coefficients, _, rank, singular = np.linalg.lstsq(a, y, rcond=None)
```

Using `lstsq` avoids explicitly computing $(A_m^T A_m)^{-1}$. This is useful because high-order power-basis matrices can be poorly conditioned. The returned rank and singular values provide numerical diagnostics. No ridge or lasso penalty is added; the model order is controlled by the Schwartz criterion.

Order selection and tie rule

The empirical risk is the training mean squared error, and the degrees of freedom count the fitted coefficients:

$$R_{\text{emp}}(m) = \frac{1}{n} \sum_{i=1}^{n} [y_i - \widehat{f}_m(x_i)]^2, \tag{6}$$

$$\text{DoF} = m + 1, \qquad p = \frac{m+1}{n}, \tag{7}$$

$$R_{\text{SC}}(m) = \left(1 + \frac{p \ln n}{1 - p}\right) R_{\text{emp}}(m). \tag{8}$$

I use this course-specific multiplicative Schwartz formula. Its numerical calculation is:

Listing 8: Computing the analytic Schwartz score.

```
p = dof / n
score = (1 + p*np.log(n)/(1-p))*training_mse
```

The condition DoF $< n$ keeps the denominator positive. With $n = 10$, I therefore search $m = 0, \ldots, 8$. I fit every candidate and select the minimum score separately for each family. The following code presents the core search, with numerical diagnostics omitted from the displayed table:

Listing 9: Fitting candidate orders and selecting the minimum Schwartz score.

```
n = len(x)
models, rows = [], []
for m in range(n-1):
    model = fit_polynomial(x, y, m, family)
    mse = float(np.mean((np.asarray(y)-model.predict(x))**2))
    rows.append({
        'm': m,
        'schwartz_criterion': schwartz(mse, m+1, n)
    })
    models.append(model)
table = pd.DataFrame(rows)
best = int(np.argmin(table.schwartz_criterion.to_numpy()))
selected_model = models[best]
```

Here `fit_polynomial` performs the least-squares fit described above, `model.predict(x)` evaluates the fitted basis expansion, and `schwartz` computes the displayed criterion. Candidate orders are evaluated in ascending order, and `argmin` returns the first minimum. Exact score ties therefore favor the smaller m. The comparison uses the computed scores without an additional near-tie tolerance.

Results and visual interpretation

Family	Selected m	DoF	Training MSE	Schwartz score
Cosine	0	1	0.10764	0.13518
Algebraic	1	2	0.08084	0.12738

The selected models are

$$\widehat{f}^{\cos}(x) = 0.63373, \qquad \widehat{f}^{\mathrm{alg}}(x) = 0.29587 + 0.56646x.$$

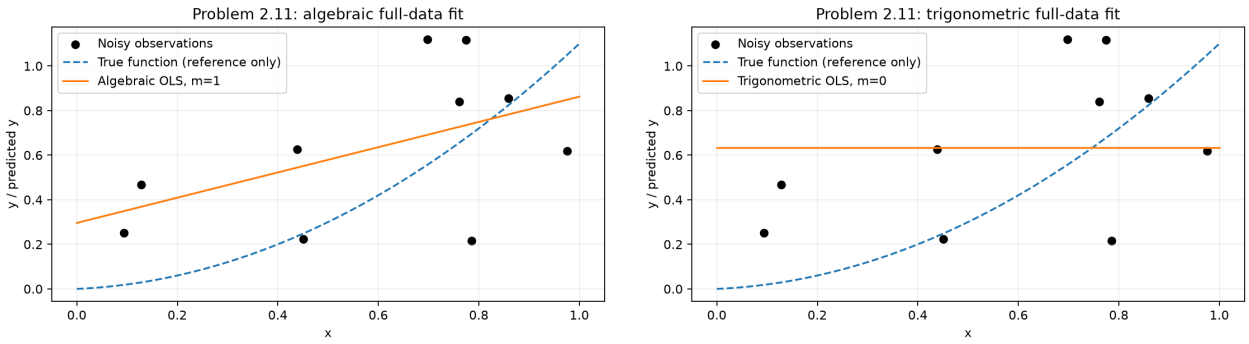
The cosine fit is a horizontal line, while the algebraic fit is linear.

Discussion and answer to the comparison question

The true generating function is quadratic, but the selected algebraic model has $m = 1$. The following comparison explains why:

Algebraic m	Training MSE	Schwartz score
1	0.08084	**0.12738**
2	0.07822	0.15541

Adding the quadratic term slightly reduces training error, but the larger complexity multiplier raises the score. With only ten noisy observations, the criterion need not recover the generating polynomial's degree.



(a) Algebraic: selected $m = 1$.

(b) Cosine: selected $m = 0$.

Figure 3: The same ten observations are used in both fits. The dashed curve is the generating function, and the solid curve is the selected least-squares model.

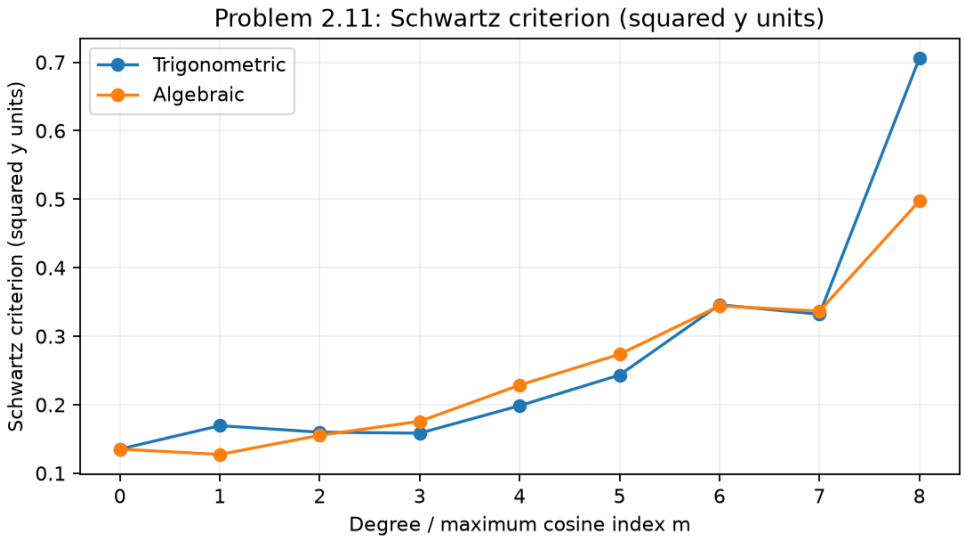


Figure 4: Schwartz scores over the candidate orders. Each family selects its own minimum; blue denotes cosine and orange denotes algebraic regression.

The cosine family also has a structural restriction. For every integer j, $\cos(2\pi j(1-x)) = \cos(2\pi jx)$, so every cosine fit satisfies $f(x) = f(1-x)$. The generating function $x^2 + 0.1x$ does not have this symmetry. Increasing m adds flexibility within this restricted family, but does not remove the restriction.

Can the best predictive family be determined from model-selection results alone? The smaller full-sample Schwartz score favors the algebraic fit under this criterion. It provides an analytic basis for a preference, but it does not establish that the algebraic method has lower prediction error on unseen observations. Both scores are computed from the same fitting sample, and neither is an observed test error. Problem 2.12 therefore compares the complete fitting-and-selection procedures using held-out observations.

Problem 2.12: Five-fold evaluation of the regression methods

Data and validation settings

I reuse the same ten observations from Problem 2.11. As required by the problem, I compare the prediction errors of the two regression methods using five-fold cross-validation, while retaining the Schwartz criterion for order selection.

The fold assignment is generated once and shared by both families:

Listing 10: Creating one shared set of five validation folds.

```
folds = list(KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
).split(x))
```

Five folds give eight training observations and two validation observations in each round. Shuffling forms the folds from a permutation of the sample order, and `random_state=42` fixes that permutation. Sharing the folds makes the two methods' errors comparable on the same held-out observations. The fold seed is separate from the seed used to generate the data, even though both are set to 42.

	Sample ID (original order)									
Fold	0	1	2	3	4	5	6	7	8	9
1	T	**V**	T	T	T	T	T	T	**V**	T
2	**V**	T	T	T	T	**V**	T	T	T	T
3	T	T	**V**	T	T	T	T	**V**	T	T
4	T	T	T	T	**V**	T	T	T	T	**V**
5	T	T	T	**V**	T	T	**V**	T	T	T

Figure 5: Actual fold assignments. T denotes training and V denotes validation. Each sample is validated once. The two V observations in a row do not participate in fitting or selecting m for that row.

Fit and select the order within each training fold

For each fold and each family, I fit candidate models using only the eight training observations, select m using their training-only Schwartz scores, and then predict the two validation observations. The training size in the criterion is now $n = 8$, rather than 10. The condition $m + 1 < 8$ gives the search range $m = 0, \ldots, 6$.

The following excerpt shows the essential calculation; recording and plotting commands are omitted:

Listing 11: Selecting the order inside each training fold before predicting validation responses.

```
for fold, (train, validation) in enumerate(folds, 1):
    for family in FAMILIES:
        model, selection = select_polynomial(
            x[train], y[train], family
        )
        predicted = model.predict(x[validation])
        mse = float(np.mean((y[validation]-predicted)**2))
```

Here `FAMILIES` contains the cosine and algebraic families. The selection procedure fits all allowed orders and returns the model with the smallest Schwartz score, using the same exact-tie rule as in Problem 2.11. It receives only `x[train]` and `y[train]`. The validation responses are used only to calculate the final error for that fold. The full-sample orders from Problem 2.11 are not reused.

For validation set V_r and training set T_r, the fold error and mean cross-validation error are

$$\mathrm{MSE}_r = \frac{1}{2} \sum_{i \in V_r} \left[y_i - \widehat{f}_{T_r}(x_i) \right]^2, \qquad \text{CV MSE} = \frac{1}{5} \sum_{r=1}^{5} \mathrm{MSE}_r. \tag{9}$$

Each observation is predicted once by a model fitted and selected without that observation. Since all five validation sets have size two, the mean fold MSE equals the MSE over all ten out-of-fold predictions. These errors evaluate the full fitting-and-order-selection procedure with eight training observations per fit.

Results

Fold	Cosine m	Cosine MSE	Algebraic m	Algebraic MSE
1	0	0.01793	1	0.02860
2	0	0.14729	6	942.63846
3	5	0.40218	3	0.33297
4	0	0.24554	0	0.24554
5	0	0.20666	0	0.20666

Family	Mean validation MSE	Sample SD across folds
Cosine	0.20392	0.14040
Algebraic	188.69045	421.46977

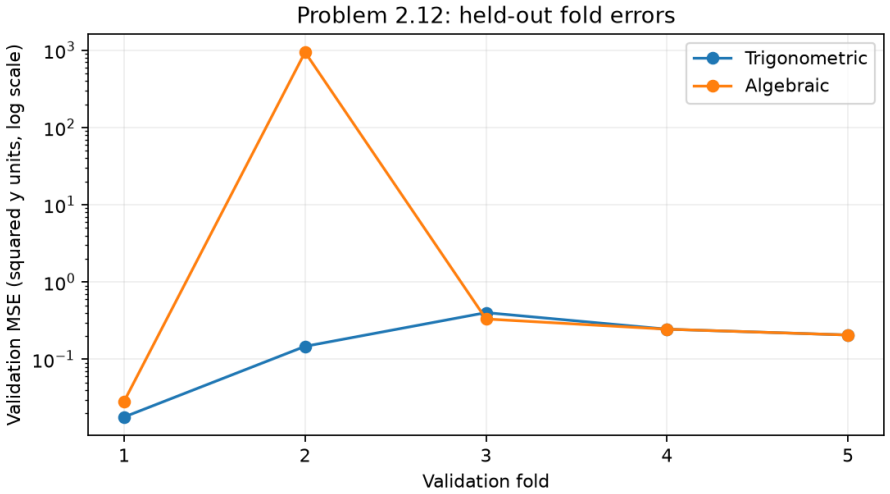


Figure 6: Held-out error by fold. The logarithmic vertical axis reveals both the ordinary errors and the very large algebraic error in fold 2.

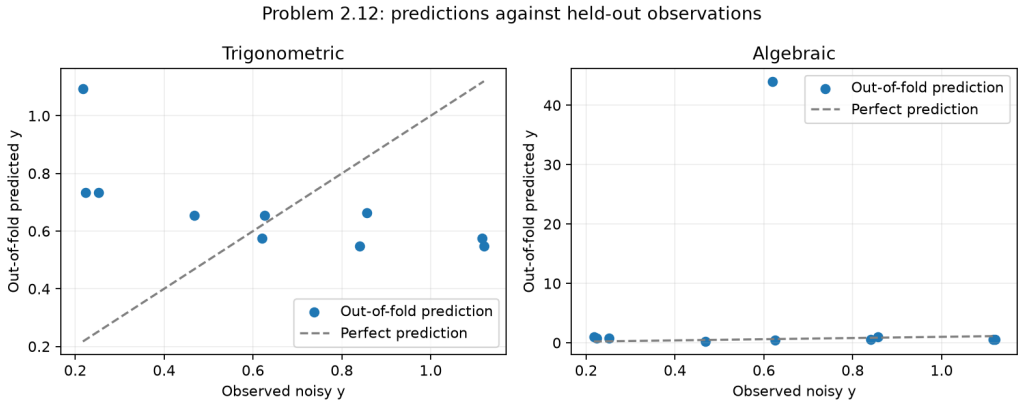


Figure 7: Out-of-fold predictions versus observed responses. The dashed diagonal indicates perfect prediction. The panels use different vertical scales: the algebraic panel exposes the extreme prediction rather than clipping it.

Discussion

The cosine procedure has the lower mean validation MSE in this particular split. The very large algebraic mean is dominated by fold 2, where the training-only criterion selects $m = 6$, with seven fitted coefficients and only eight training observations.

The training inputs in that fold range from approximately 0.09418 to 0.85860. One held-out input, $x = 0.97562$, lies beyond this range. The selected algebraic model predicts 44.03538 at this point, while the observed response is 0.61975. Its other held-out prediction is approximately 0.51596 for an observed response of 1.11610. These two errors give

$$\mathrm{MSE}_2^{\mathrm{alg}} \approx \frac{(44.03538 - 0.61975)^2 + (0.51596 - 1.11610)^2}{2} = 942.64.$$

This is an example of a high-order fit extrapolating poorly beyond its training-input range. The selected design matrix has full column rank, but its condition number is approximately 5.31×10^5,

indicating numerical sensitivity. Using a least-squares solver that avoids explicit matrix inversion does not prevent a flexible model from making extreme held-out predictions. The training criterion penalizes complexity, but it does not guarantee reliable predictions for every small training sample.

The cosine method selects the constant model in four folds and $m = 5$ in one fold. In folds 4 and 5, both families select $m = 0$ and therefore produce identical predictions and errors within each fold. The larger algebraic mean should be reported with its fold-level results so that its dependence on one extreme fold is visible.

The sample standard deviation across folds describes the spread of the five fold errors; it is not a confidence interval or the standard error of the mean. With only ten observations, two validation observations per fold, and one fixed split, the comparison is sensitive to which observations are held out. The result supports the cosine procedure for this experiment, but does not establish that it is universally better.

Relation to Problem 2.11. The full-sample Schwartz criterion favors the algebraic family, whereas held-out MSE in this five-fold evaluation favors the cosine procedure. The first comparison uses an analytic selection score on all ten observations; the second measures held-out errors after fitting and selecting on eight observations. They answer different questions, and the difference illustrates why the model-selection scores alone do not settle the predictive comparison.

Summary of Results

Problem	Experimental design	Main result
2.7	Train on 2004; test on 2000	$k = 7$; test accuracy 49.02%
2.8	Train on 2000; test on 2004	$k = 7$; test accuracy 56.86%
2.11	Select m using Schwartz	Algebraic $m = 1$; cosine $m = 0$
2.12	Five-fold evaluation	Mean MSE: cosine 0.20392; algebraic 188.69045

Conclusion. Problem 2.11 favors the algebraic family by its full-data criterion value, while Problem 2.12 favors the cosine procedure by held-out MSE. These are different assessments. Ten observations and one five-fold split provide limited evidence; the results do not establish universal superiority of either family. For KNN, the cross-year results likewise show the limits of obesity as a single predictor of the supplied election labels.